

Disciplina/Tema: Agentes Conversacionais

Professor(a): Yuri Lopes Queiroz

🚀 9又 = DR Á) - ? 也 [- { D } & ≈
也 a R ? 7 ! { Î { Y -

Arquitetura Profissional com LLMs Reais

MBA IA Generativa PCDF/IBMEC | Terça, 04 de Julho de 2026

Jornada E1 → E5

Cada etapa construiu a base para a arquitetura profissional que chegamos hoje.

01

E1 — Anatomia do Agente

Fundamentos estruturais de um agente de IA

02

E2 — Qualidade e Memória

Mecanismos de retenção e melhoria de respostas

03

E4 — RAG + FAISS

Recuperação aumentada com indexação vetorial

04

E5 — Especialização

Fine-tuning com LoRA e processamento multimodal



Onde Estamos: E6

✓ O que já conquistamos

- Agente funcionando perfeitamente
- RAG especializado e indexado
- Fine-tuning com LoRA
- Processamento multimodal

🏭 O Desafio de E6

Temos um agente poderoso — mas **como colocar em PRODUÇÃO com segurança?**

É hora de construir a arquitetura profissional completa.

O Problema: Do Desenvolvimento para Produção

Sistemas de IA em produção enfrentam ameaças reais que o ambiente de desenvolvimento não revela.

✗ Entrada Maliciosa

SQL injection e command injection podem comprometer toda a infraestrutura do sistema.

✗ Abuso de Requisições

Múltiplas requisições e ataques DDoS podem derrubar o serviço por completo.

✗ Vazamento de PII

Exposição de dados pessoais viola a LGPD e gera responsabilidade jurídica grave.

✗ Linguagem Tóxica

Respostas agressivas ou discriminatórias destroem a credibilidade do produto.

⚠ Como proteger o sistema em produção? A resposta está na arquitetura profissional de E6.

Solução: Arquitetura Profissional em 4 Camadas



1

Camada 1 — Guardrails de Entrada

Proteção e validação de toda entrada do usuário antes de processar

2

Camada 2 — LangGraph

Orquestração de fluxo complexo e iterativo entre componentes

3

Camada 3 — Multi-Agentes

Execução distribuída com LLMs reais e agentes especializados

4

Camada 4 — Guardrails de Saída

Validação e filtragem da resposta antes de entregar ao usuário



Guardrails: Proteção em Camadas

Definição

Mecanismos de segurança que controlam o comportamento de IA, **validando entrada e saída** para evitar comportamentos indesejados em produção.

1	Input Validation Valida formato e conteúdo da entrada
2	Rate Limiting Controla número de requisições
3	PII Detection Detecta dados pessoais (LGPD)
4	Output Filtering Filtra respostas perigosas
5	Toxicity Filtering Remove linguagem tóxica

Fluxo com Guardrails

Cada camada de validação pode aprovar (✓) ou bloquear (X) o fluxo — garantindo segurança em toda a pipeline.



O fluxo de Guardrails garante que **nenhuma entrada maliciosa** chega ao LLM e **nenhuma resposta perigosa** chega ao usuário final.



LangGraph: Orquestração de Fluxo

Definição

Framework para criar **grafos de execução com LLMs**, permitindo fluxo de controle complexo e iterativo entre múltiplos componentes.

- ❏ Permite orquestração complexa e iterativa de múltiplos agentes com decisões dinâmicas em tempo real.

Nodes

Funções ou agentes que executam tarefas específicas no grafo

Edges

Conexões entre nodes que definem a direção do fluxo

State

Dados compartilhados e persistentes entre todos os nodes

Conditional Edges

Decisões dinâmicas — o próprio LLM decide o próximo passo

Multi-Agentes: Orquestração Inteligente

Definição: Sistema com múltiplos agentes (cada um com LLM próprio), coordenados por um Supervisor para resolver problemas complexos de forma distribuída.



Supervisor (LLM)

Coordena e distribui tarefas entre os agentes especializados



Agent 1 (LLM)

Especialista em análise de dados e interpretação de contexto



Agent 2 (LLM)

Especialista em processamento e transformação de informações

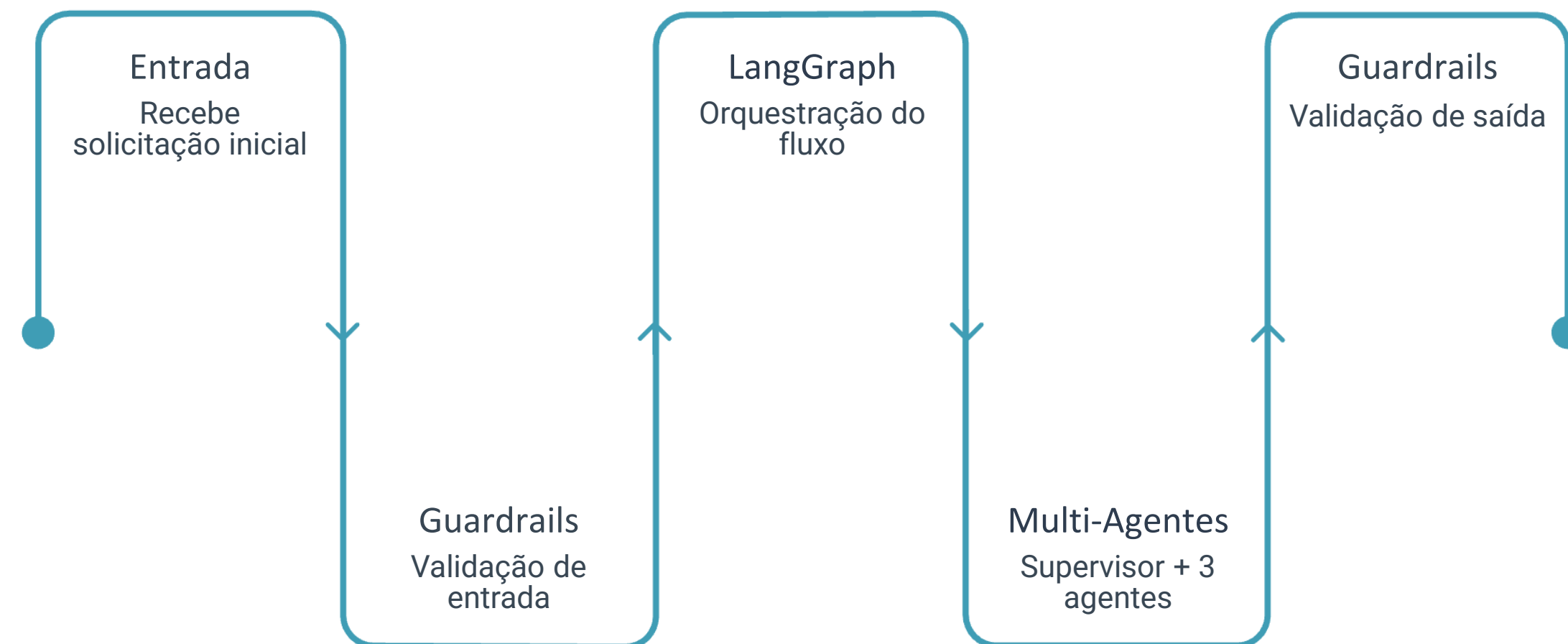


Agent 3 (LLM)

Especialista em geração e formatação da resposta final



Arquitetura Final: Tudo Junto



Por que essa arquitetura funciona?

Segurança em dupla camada

Guardrails protegem tanto a entrada quanto a saída, sem pontos cegos

Orquestração inteligente

LangGraph garante fluxo correto mesmo em cenários complexos e iterativos

Especialização distribuída

Cada LLM faz exatamente o que sabe melhor — máxima qualidade de saída

Pronto para produção

Arquitetura robusta, escalável e em conformidade com LGPD

Guardrail 1: Input Validation

Primeira linha de defesa contra entradas maliciosas — bloqueando ataques antes de chegarem ao sistema.

O que protege

- ✓ SQL Injection
- ✓ XSS (Cross-Site Scripting)
- ✓ Command Injection
- ✓ Path Traversal

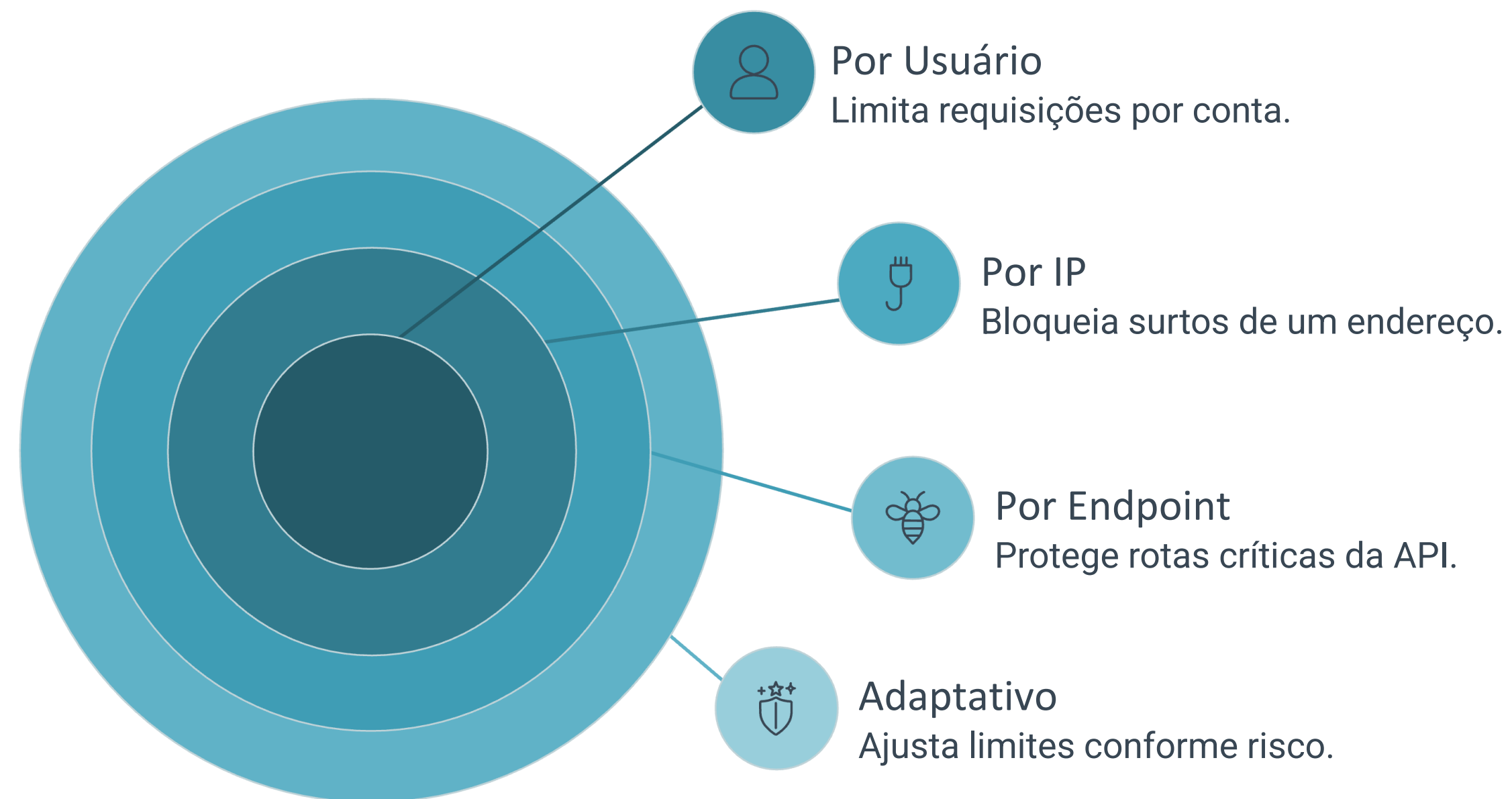
Exemplos bloqueados

- ✗ `' ; DROP TABLE users; --`
- ✗ `<script>alert('xss')</script>`
- ✗ `rm -rf /`

Implementação

Regex + Whitelist — padrões perigosos são identificados e rejeitados antes de qualquer processamento.

Rate Limiting



O algoritmo **Token Bucket** controla o fluxo de requisições, garantindo que nenhum usuário ou IP possa sobrecarregar o sistema.

O que protege

- ✓ Abuso (múltiplas requisições)
- ✓ DDoS (ataque de negação de serviço)
- ✓ Força bruta
- ✓ Scraping

Estratégias de Aplicação

Por Usuário

Por IP

Por Endpoint

Adaptativa

PII Detection

Dados Pessoais Detectados

✓ CPF

123.456.789-00

✓ Email

usuario@example.com

✓ Telefone

(11) 98765-4321

✓ Cartão de Crédito

Números e CVV mascarados

✓ RG / Passaporte

Documentos de identidade

Fluxo de Proteção



Conformidade com a **LGPD** – Lei Geral de Proteção de Dados Pessoais exige detecção tanto na entrada quanto na saída.

GUARDRAIL 4

Output Filtering

Mesmo que uma entrada passe pelos guardrails anteriores, a **saída** precisa ser inspecionada antes de chegar ao usuário.



PII (CPF, Email, Telefone)

Dados pessoais são removidos ou mascarados da resposta gerada.



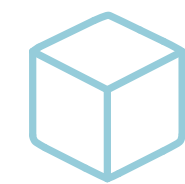
Informações Sensíveis

Segredos, senhas e dados internos não devem vaziar na saída.



URLs Suspeitas

Links maliciosos ou de phishing são filtrados antes da entrega.



Comandos de Código

Outputs com comandos executáveis são bloqueados por segurança.

Entrada: "Seu CPF é 123.456.789-00" → Saída filtrada: **"Seus dados foram registrados"**

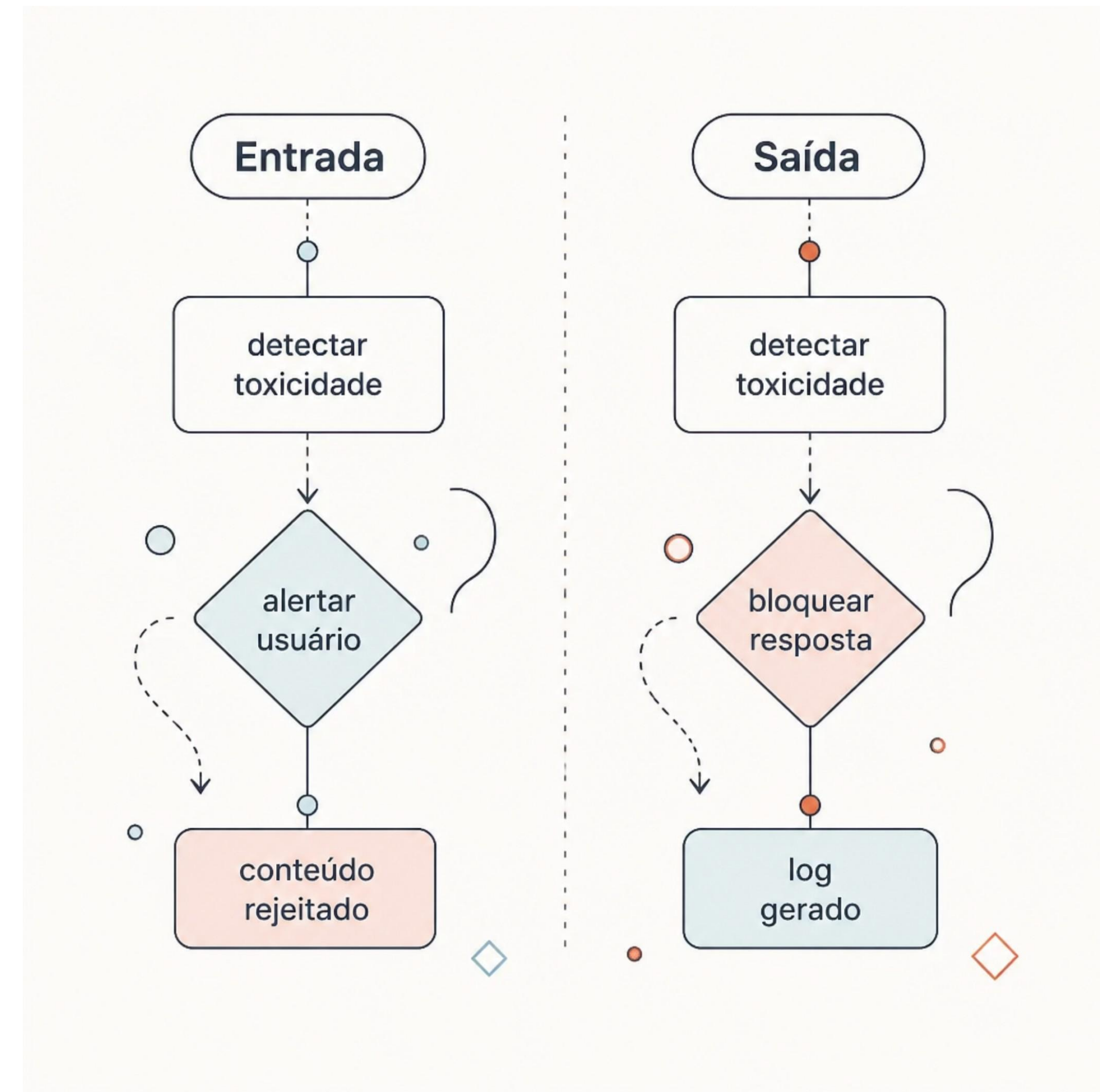
Toxicity Filtering

Tipos de Toxicidade Detectados

- ✓ Xingamentos e insultos
- ✓ Discriminação (raça, gênero, religião)
- ✓ Ameaças e violência
- ✓ Assédio sexual
- ✓ Spam

Implementação

Combinação de três camadas: **palavras-chave**, **padrões regex** e **modelos de ML** para detecção contextual.



LangGraph: Nodes e Edges

Conceitos Fundamentais

NODE

Função ou Agente que executa uma tarefa específica no grafo.

EDGE

Conexão entre nodes – define por onde os dados fluem.

STATE

Dados compartilhados e acessíveis entre todos os nodes.



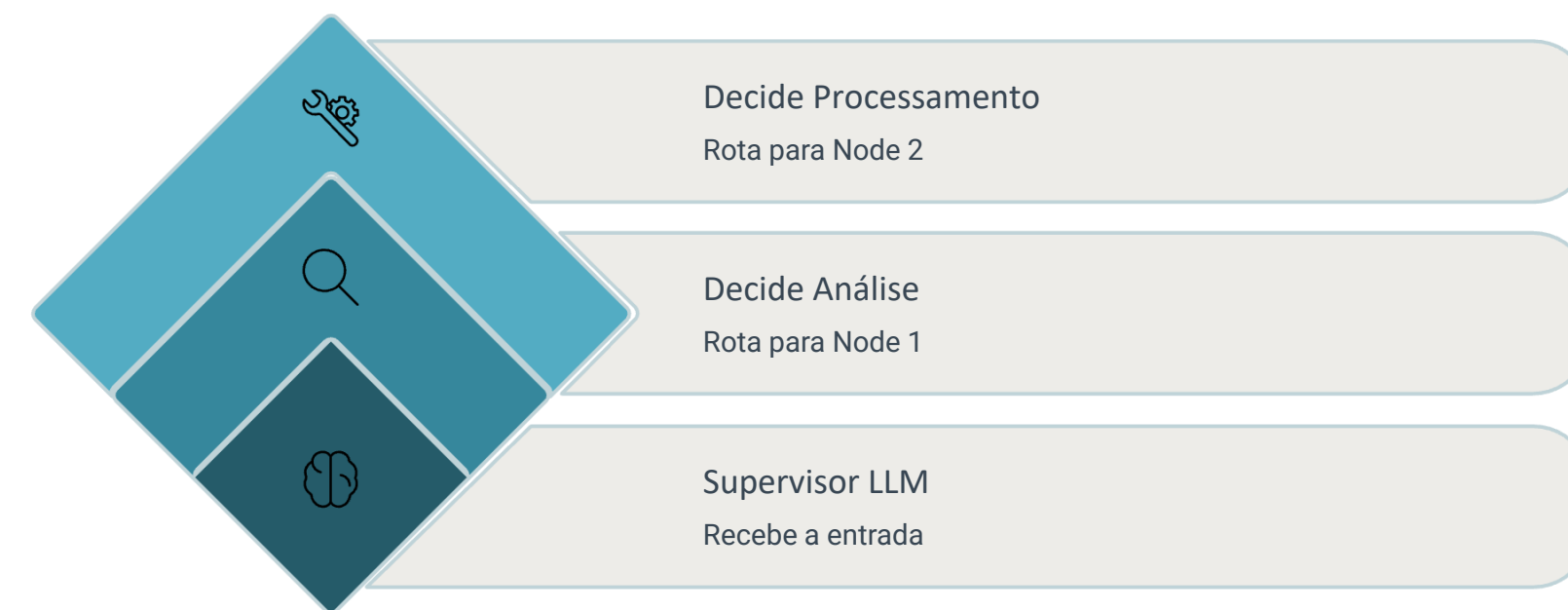
Análise

Processamento

Resposta

Conditional Edges

O LLM decide qual node executar — sem determinismo fixo



Por que isso importa

- Entrada chega ao Supervisor (LLM)
- LLM avalia: "Preciso analisar dados"
- Rota para Node 1 (Análise)
- LLM avalia: "Agora preciso processar"
- Rota para Node 2 (Processamento)

❏ Sem determinismo — o LLM **pensa e decide** o caminho em tempo real.

State: Dados Compartilhados

O **State** é o repositório central do grafo — cada node lê do state, executa sua tarefa, e escreve o resultado de volta.

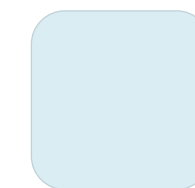
```
State = {  
  'input':      'Analise isto...',  
  'analise':    'Resultado da análise',  
  'processamento': 'Resultado do processamento',  
  'resposta':   'Resposta final'  
}
```

Características do State



Leitura e Escrita

Cada node pode ler qualquer campo do state e escrever novos resultados.

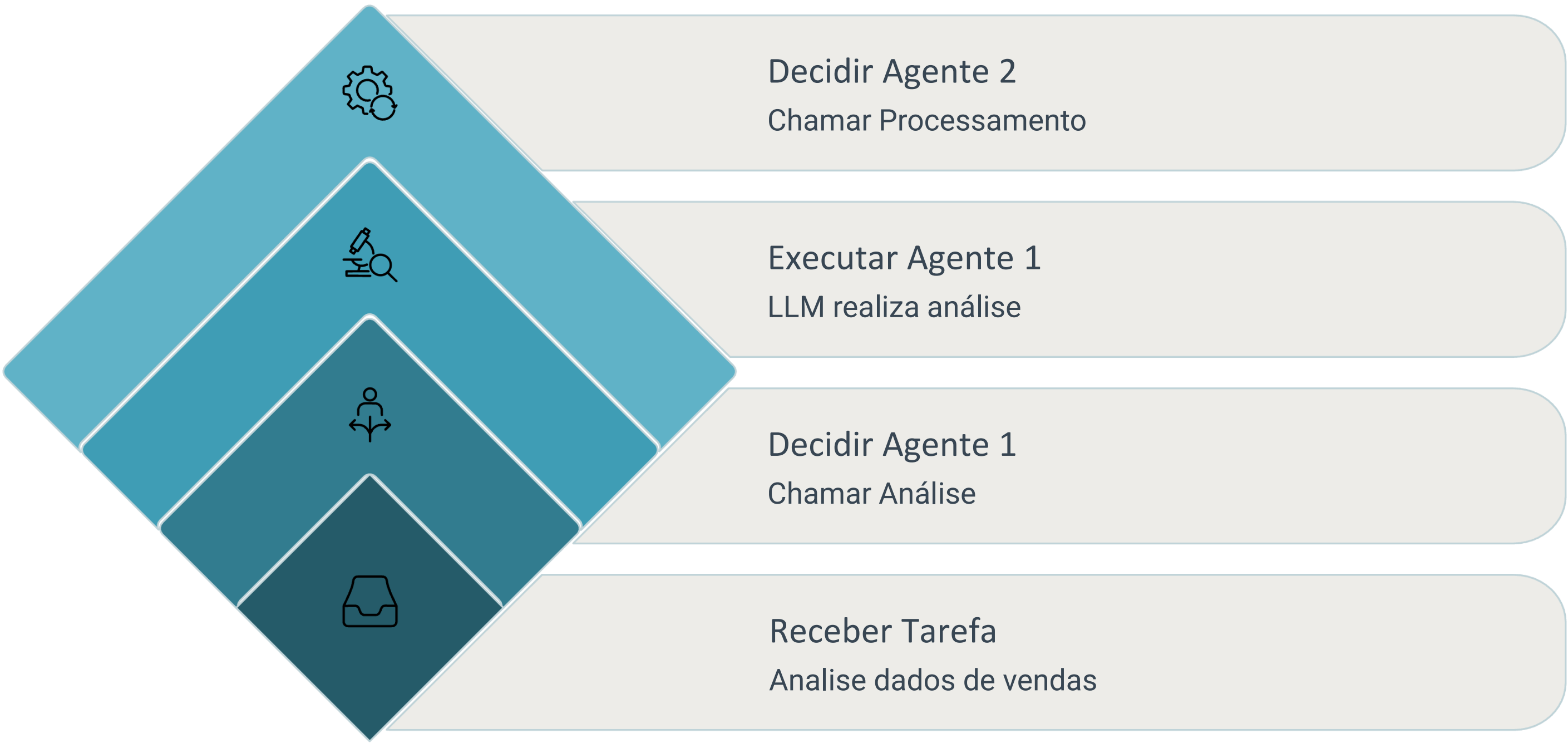


Histórico Completo

Todo o histórico de execução fica disponível para nodes subsequentes.

Supervisor: LLM Coordena

O **Supervisor** é um LLM que recebe a tarefa, decide qual agente especializado chamar, recebe o resultado e decide o próximo passo.



 Supervisor (LLM) Orquestra e decide	 Agent1 (LLM) Executa análise
 Agent2 (LLM) Executa processamento	 Resultado Final Entregue ao usuário

Workers: Agentes Especializados

📄 Cada agente possui um **LLM REAL** com ferramentas e especialização próprias — não são funções genéricas.



Agent 1: Análise

LLM especialista em análise de dados

- Análise estatística
- Identificação de padrões



Agent 2: Processamento

LLM especialista em processamento

- Transformação de dados
- Limpeza e normalização



Agent 3: Resposta

LLM especialista em comunicação

- Formatação de output
- Resumo executivo

Multi-Agentes: Comunicação Entre Agentes

Fluxo de Comunicação via LangGraph — arquitetura onde um Supervisor LLM orquestra agentes especializados em sequência coordenada



Supervisor (LLM)

Recebe a entrada e decide qual agente chamar baseado no estado atual e no contexto da tarefa



Agent 1 (LLM)

Processa tarefa especializada e retorna resultado ao Supervisor para próxima decisão



Agent 2 (LLM)

Próxima tarefa conforme decisão do Supervisor — cada agente é um LLM especializado



Orquestração

Tudo coordenado via LangGraph — fluxo baseado em estado compartilhado e entrada do usuário

- ❏ Cada agente é um LLM especializado. O Supervisor coordena o fluxo baseado em estado e entrada — nunca executa tarefas diretamente.

Multi-Agentes: Exemplo Prático PCDF

Cenário

Entrada: "Analise dados de armas apreendidas"

Sistema PCDF — Polícia Civil do Distrito Federal

Saída Esperada

Relatório estruturado com insights, métricas e padrões identificados automaticamente

1

Agent 1 — Análise

Extraí padrões dos dados brutos de armas apreendidas: tipos, datas, regiões, frequências

2

Agent 2 — Processamento

Calcula métricas e agregações: totais por categoria, tendências temporais, distribuição geográfica

3

Agent 3 — Resposta

Gera relatório estruturado com conclusões, visualizações textuais e recomendações operacionais

Prática: Setup Jupyter Notebook

Seis etapas progressivas que constroem um sistema multi-agentes completo e funcional com guardrails de produção



Configurar LLM

Ollama (local) ou OpenRouter (fallback)



Testar Conexão

Verificar provider ativo em tempo real



Guardrails

Input Validation, Rate Limiting, PII Detection



Grafo LangGraph

3 nodes com conditional edges dinâmicos



3 Agentes

Análise, Processamento e Resposta especializados



Pipeline Completo

Integração total com dados PCDF reais

📄 **Resultado final:** Sistema multi-agentes funcional com guardrails — código real, não simulado. Alunos veem tudo funcionando ao vivo.

Setup e Configuração do Ambiente

Passos de Configuração

1 Instalar Dependências

`langchain`, `langgraph`, `ollama`, `openrouter` — ambiente complet

2 Configurar Ollama Local

Conectar em `localhost:11434` — modelo rodando localmente, sem custo por token

3 Configurar OpenRouter

API key configurada como fallback automático quando Ollama não estiver disponível

Lógica de Fallback Automático

Testar Ollama

Verificar se modelo está rodando em localhost — ping inicial antes de qualquer requisição

Fallback Inteligente

Se Ollama falhar → switch automático para OpenRouter sem interrupção do fluxo

Exibir Provider Ativo

`print(f"LLM ativo: {provider}")` — alunos confirmam qual ambiente está sendo usado

L} &^ } ^ (7)DR Á } → ?

Três camadas de proteção essenciais aplicadas antes de qualquer entrada chegar ao LLM



L{&R7e- Á 7-ç

Verificar formato e tamanho da entrada.
Entradas malformadas, injeções de prompt ou textos excessivamente longos são bloqueados imediatamente.



w-Ŷ [→ -7ê

Limitar requisições por minuto por usuário.
Protege o sistema contra sobrecarga e uso abusivo em produção policial.



t LL5Î Ŷ€7-ç

Detectar e mascarar dados sensíveis:
CPF, RG, nomes de investigados, endereços. Conformidade com LGPD.

⊘ 9 (7)Á a - € → -

Detectada pelo Input Validation →
Bloqueada antes de chegar ao LLM. Log registrado para auditoria.

✓ 9 (7)Á ë "Á

Passa pelos 3 guardrails sem bloqueios
→ **Encaminhada ao LLM** para processamento normal.

🔒 5-Á < -{Î (-Ŷ →

PII Detection identifica CPF, RG →
Mascarados automaticamente antes do processamento.

Criar Grafo LangGraph

Estrutura dos 3 Nodes

Node 1 — Análise

LLM analisa entrada e extrai padrões: categorias de armas, datas, regiões geográficas

Node 2 — Processamento

LLM processa dados extraídos e calcula métricas: totais, tendências, distribuições estatísticas

Node 3 — Resposta

LLM gera resposta estruturada com conclusões, insights e recomendações operacionais

Conditional Edges

Decisão Dinâmica

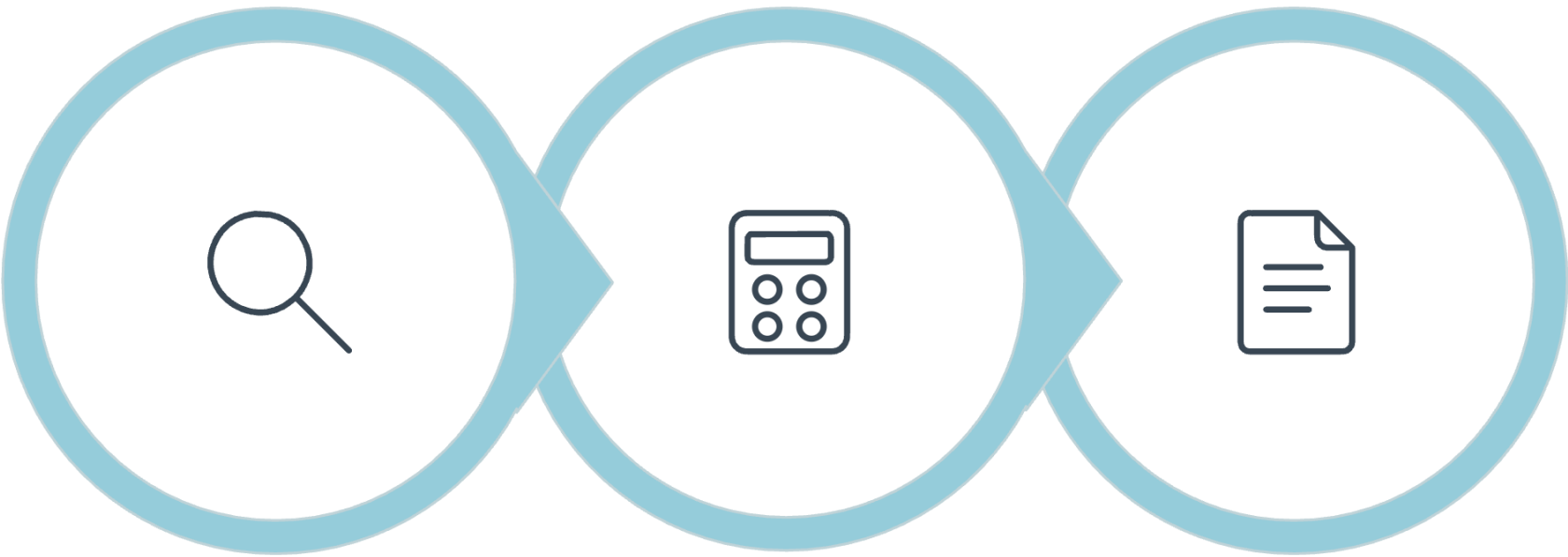
LLM decide qual node executar baseado no estado atual — não há sequência fixa obrigatória

Fluxo Adaptativo

Resultado de cada node determina o próximo passo — loops e branches são suportados nativamente

Visualização Real-Time

Grafo visualizado em tempo real durante execução — alunos veem com dados PCDF reais



Análise

Processamento

Resposta

Criar Multi-Agentes Especializados

Três agentes LLM, cada um com prompt e especialização distintos, coordenados pelo Supervisor



Agent 1 — Análise

LLM especialista em análise de padrões. Prompt otimizado para identificar tendências em dados de ocorrências, armas e registros operacionais da PCDF.



Agent 2 — Processamento

LLM especialista em cálculos e métricas. Agrega totais, calcula distribuições e produz estatísticas estruturadas prontas para relatório.



Agent 3 — Resposta

LLM especialista em geração de relatórios. Transforma outputs dos agentes anteriores em documentos estruturados com linguagem técnica policial.

Supervisor (LLM)

Coordena qual agente chamar e em qual ordem. Mantém estado compartilhado entre chamadas. Decide quando o resultado final está pronto.

Resultado Observável

- Agentes conversando entre si via estado compartilhado do LangGraph
- Orquestração inteligente via Supervisor — sem hardcode de sequência
- Cada agente especializado e focado exclusivamente em sua tarefa

LLM, com validação de entrada, limitação de taxa e detecção de PII antes de chegar ao LLM



DR ->

D -> [- D]

a R -> L

W -> 9 -> R -> A

/ -> & L -> L -> A

DR ->

Input Validation + Rate Limiting + PII Detection – 3 camadas ativas antes do LLM

[- D]

Grafo de orquestração com 3 nodes e conditional edges dinâmicos

a R -> L

3 agentes especializados + Supervisor coordenando execução completa

9 -> & t -> W -> C R -> / -> & 7

9 -> A

Dados brutos de armas apreendidas – registros operacionais da PCDF

t -> L -> L

Guardrails -> LangGraph -> Agentes em sequência orquestrada

{ -> A

Relatório estruturado com padrões, métricas e recomendações operacionais

Demonstração ao vivo – alunos veem o pipeline completo funcionando em tempo real com dados reais da PCDF.

Projeto Final: Arquitetura de Produção

Estrutura de Diretórios — Produção

```
src/
├── guardrails/
│   ├── input_validation.py
│   ├── rate_limiting.py
│   ├── pii_detection.py
│   ├── output_filtering.py
│   └── toxicity_filtering.py
├── langgraph/
│   └── orchestration_graph.py
├── agents/
│   ├── analysis_agent.py
│   ├── processing_agent.py
│   └── response_agent.py
└── api.py ← API REST
```

Módulos por Camada

guardrails/ — 5 arquivos

Input, Rate, PII, Output Filtering e Toxicity — proteção completa de entrada e saída

langgraph/ — Orquestração

Graph único com todos os nodes, edges e estado compartilhado entre agentes

agents/ — 3 especialistas

Analysis, Processing e Response — cada agente em arquivo próprio, testável isoladamente

api.py — REST Endpoint

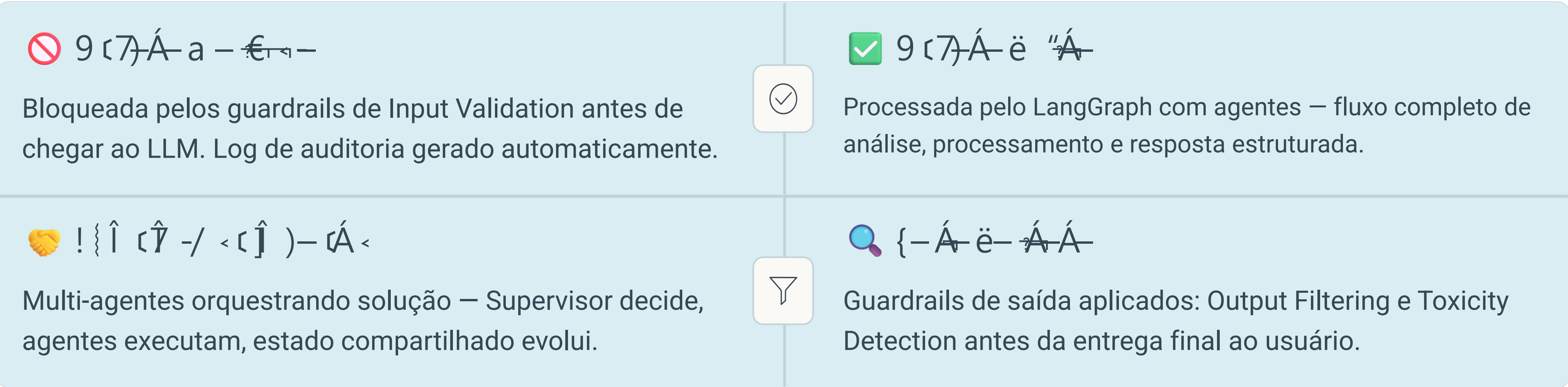
Interface de produção — expõe o pipeline completo como API consumível por sistemas PCDF

Casos PCDF Implementados

- Análise de armas apreendidas por tipo, data e região
- Identificação de padrões criminais em séries temporais
- Geração automática de relatórios estruturados operacionais

5Î } < [-7), ' < - < ë -] < 7A/Î [") -

Alunos veem tudo funcionando em tempo real – cada cenário demonstra uma camada diferente do sistema



 ! [" Î Á !) } - -

Dados de armas apreendidas processados automaticamente com extração de padrões por tipo e região

 t - Á) \$ Î - /) } - € -

Detecção de padrões em séries temporais de ocorrências – análise preditiva para operações futuras

 w Î ÷ 7 =) - ! R 7 < } 7 € < -

Geração de documentos estruturados prontos para uso operacional pela PCDF sem intervenção manual

Projeto Final: Caso PCDF 1 — Análise de Armas

ibmec.br

ENTRADA

"Quantas armas foram apreendidas em julho?"

Fluxo de Processamento

01

Guardrails validam entrada

02

LangGraph orquestra

03

Agent 1 (Análise): Extrai dados

SAÍDA

Saída do sistema vai aqui.

Projeto Final: Caso PCDF 1 — Análise de Armas (Continuação)

Para finalizar o fluxo de processamento e a arquitetura geral:

- Agent 2 (Processamento): Calcula
- Agent 3 (Resposta): Gera resposta
- Guardrails validam saída

Saída: "Em julho foram apreendidas X armas..."



Arquitetura de produção: Guardrails → LangGraph → Multi-Agentes → Guardrails

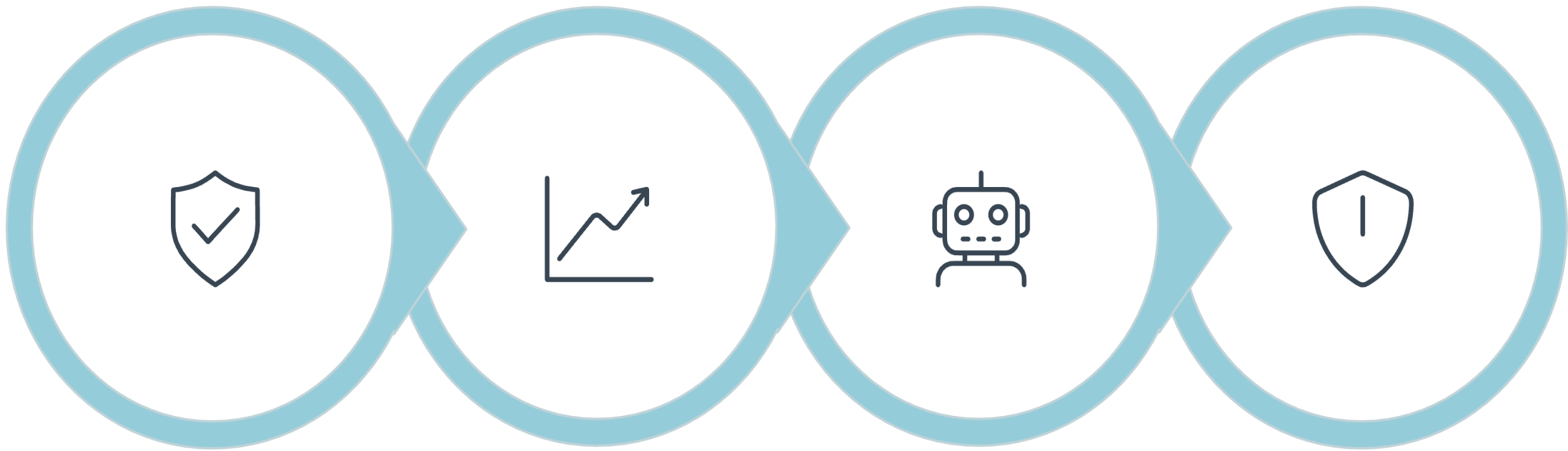
Projeto Final: Caso PCDF 2 — Relatório de Ocorrências



ENTRADA

"Gere relatório de ocorrências por tipo"

Fluxo de Processamento



Guardrails
Entrada

LangGraph
Orquestra

Agentes
Processam

Guardrails
Saída

SAÍDA

Relatório estruturado com ocorrências categorizadas

Projeto Final: Caso PCDF 3 — Análise de Tendências

ENTRADA

"Quais são as tendências criminais?"

Fluxo de Processamento

- 1

Guardrails
Validam entrada
- 2

LangGraph
Orquestra
- 3

Agentes
Analisam padrões
- 4

Guardrails
Validam saída

SAÍDA

Análise de tendências com insights acionáveis



Projeto Final: Estrutura de Código

Código profissional, pronto para produção

Guardrails

```
src/guardrails/in  
put_validation.py
```

```
src/guardrails/ra  
te_limiting.py
```

```
src/guardrails/pi  
i_detection.py
```

```
src/guardrails/ou  
tput_filtering.py
```

```
src/guardrails/tox  
icity_filtering.py
```

Orquestração e Agentes

```
src/langgraph/gra  
ph.py
```

```
src/agents/agent_  
analise.py
```

```
src/agents/agent_  
processamento.py
```

```
src/agents/agent_  
resposta.py
```

```
src/api.py
```

Projeto Final: Testes Automatizados



Cobertura Completa

tests/test_guardrails.py

Validação de proteções

tests/test_langgraph.py

Orquestração

tests/test_agents.py

Comportamento dos agentes

tests/test_integracao.py

Fluxo completo

✔ **Objetivo:** Validação de todos os componentes antes de produção

Boas Práticas: Produção

Checklist de Implementação

- ✓ Guardrails em Camadas
Implementar proteções em múltiplos níveis
- ✓ LangGraph
Usar para orquestração robusta
- ✓ Agentes Especializados
Criar agentes focados em tarefas únicas
- ✓ Testes Automatizados
Testar tudo antes de deploy
- ✓ Monitorar
Observar comportamento em produção
- ✓ Logar
Registrar tentativas bloqueadas
- ✓ Atualizar
Melhorar continuamente
- ✓ Documentar
Manter documentação sempre atualizada



Próximos Passos: Sua Jornada

ibmec.br



O que fazer agora

-  Revisar material em casa
-  Fazer atividades extras
-  Criar seus próprios agentes
-  Implementar guardrails
-  Deploy em produção
-  Monitorar e melhorar

Recursos e Referências

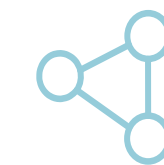


Links Úteis



LangChain

<https://langchain.com>



LangGraph

github.com/langchain-ai/langgraph



Ollama

<https://ollama.ai>



OpenRouter

<https://openrouter.ai>



LGPD

gov.br/lgpd

Documentação completa em material de apoio